FUNDAMENTOS DE TERRAFORM · CLASE 3

Resumen: Providers, Resources y State

Instructor: Ing. Álvaro Socop

1. Providers: los drivers de Terraform

Un **provider** es un plugin que permite a Terraform comunicarse con una API externa. Traduce tu código HCL en llamadas a la API para **crear, leer, actualizar y borrar** (CRUD) recursos, maneja la autenticación y define qué resources y data sources existen. Se descarga automáticamente con `terraform init`.

> **Analogía**
>
> El provider es como un **driver**: sin él, Terraform no sabe hablar con AWS, Azure o GCP. En la Clase 2 usamos `hashicorp/local` (la API era tu disco duro); hoy usamos `hashicorp/aws`. **El flujo de trabajo es exactamente el mismo.**

	Oficiales (HashiCorp / Partner)	Terceros (Community)
Quién lo mantiene	HashiCorp o la empresa dueña del servicio	La comunidad o personas individuales
Soporte y docs	Garantizados	Variables
Versionado	Estable	Puede abandonarse
Ejemplos	AWS, Azure, GCP, Kubernetes	Muchos proyectos open source y herramientas de nicho

Todos se buscan en **registry.terraform.io**. Antes de usar uno de la comunidad revisa sus descargas, la fecha del último release y su documentación.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"   # namespace/provider
      version = "~> 5.0"          # cualquier 5.x, nunca 6.0
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

Elemento	Qué significa
`source`	De dónde se descarga el provider (namespace/nombre en el Registry).
`version = "~> 5.0"`	Acepta versiones compatibles (5.1, 5.100…) pero nunca la 6.0, que trae cambios incompatibles.
`provider "aws" {}`	Configura el provider: aquí, en qué región se crean los recursos.
Credenciales	**Nunca** van en el código. Terraform las toma de `aws configure` o de variables de entorno.
`terraform init`	Descarga el provider a `.terraform/` y fija la versión exacta en `.terraform.lock.hcl`.

2. Resources y addressing

Un **resource** es un componente de infraestructura que Terraform administra. Se declara con `resource "TIPO" "NOMBRE" { ... }`: el **tipo** lo define el provider y el **nombre** lo eliges tú (solo existe dentro de Terraform).

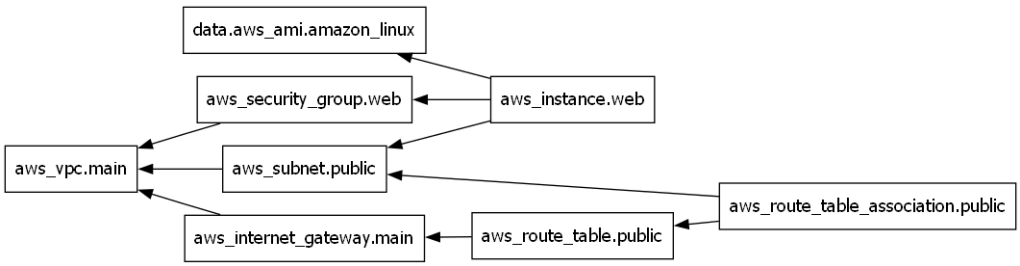
Resource	Qué es en AWS
`aws_vpc`	Red virtual privada: tu propia red aislada dentro de AWS.
`aws_subnet`	Un segmento (rango de IPs) dentro de la VPC.
`aws_security_group`	Firewall: qué tráfico puede entrar y salir.
`aws_instance`	Máquina virtual EC2 (un servidor).
`aws_s3_bucket`	Almacenamiento de archivos (objetos).

Cada resource tiene **argumentos** (lo que tú escribes, por ejemplo `cidr_block`) y **atributos** (lo que AWS devuelve al crearlo, por ejemplo `id` o `public_ip`). En el plan, los atributos aparecen como `(known after apply)`.

Terraform identifica cada recurso con una dirección única `TIPO.NOMBRE`, por ejemplo `aws_vpc.main` o `aws_instance.web`. Con esa dirección puedes usar los atributos de un recurso en otro:

```
resource "aws_subnet" "public" {
  vpc_id     = aws_vpc.main.id      # ← referencia a otro resource
  cidr_block = "10.0.1.0/24"
}
```

> **Dependencias implícitas**
>
> Al escribir `aws_vpc.main.id` dentro de la subnet, Terraform entiende que **primero debe crear la VPC**. Nunca le indicas el orden: lo deduce construyendo un **grafo de dependencias**. Al destruir, el orden se invierte.
>
> Puedes ver el grafo con `terraform graph`.



Grafo del lab de hoy: las flechas apuntan hacia el recurso del que se depende.

3. State: la única fuente de verdad

El archivo `terraform.tfstate` es la **memoria** de Terraform. Relaciona cada recurso de tu código (`aws_instance.web`) con el objeto real en AWS (`i-0abcdef1234567890`). Gracias a él, `plan` puede comparar lo que dice tu código con lo que existe en la realidad.

```
{
  "version": 4,
  "resources": [{
    "type": "aws_instance",
    "name": "web_server",
    "instances": [{
      "attributes": {
        "id": "i-0abcdef1234567890",
        "instance_type": "t2.micro"
      }
    }]
  }]
}
```

Riesgo del state local	Por qué importa
Se pierde	Terraform ya no sabe qué recursos son suyos; intentaría crearlos todos de nuevo. Habría que importarlos uno por uno.
No tiene locking	Si dos personas ejecutan `apply` al mismo tiempo, el state se corrompe.
Datos sensibles	Contraseñas, IPs y ARNs quedan en JSON sin cifrar. **Nunca se sube a Git.**
Sin historial	No sabes qué cambió ayer ni puedes volver atrás.

La solución en equipos es un backend remoto: AWS S3, Azure Blob Storage, Google Cloud Storage o Terraform Cloud / HCP. Ofrecen locking automático, cifrado en reposo y versionado.

> **Dato actualizado**
>
> Desde Terraform 1.11 el backend S3 bloquea el state por sí solo con `use_lockfile = true`. La combinación S3 + DynamoDB que verás en muchos tutoriales es la forma antigua (deprecada).

4. Lab: infraestructura web en AWS

Construimos una red completa con un servidor web. El proyecto se divide en `providers.tf`, `variables.tf`, `main.tf` y `outputs.tf`: Terraform lee **todos** los `.tf` de la carpeta como si fueran un solo archivo.

Dirección	Para qué sirve
`aws_vpc.main`	Red privada 10.0.0.0/16.
`aws_subnet.public`	Subnet 10.0.1.0/24 que asigna IP pública a sus servidores.
`aws_internet_gateway.main`	Puerta de la VPC hacia internet.
`aws_route_table.public` **+ asociación**	Envía el tráfico 0.0.0.0/0 al Internet Gateway. Sin esto, la subnet no es realmente pública.
`aws_security_group.web`	Permite entrar por SSH (22) y HTTP (80), y salir a cualquier destino.
`data.aws_ami.amazon_linux`	**Data source**: no crea nada, consulta la imagen de Amazon Linux 2023 más reciente.
`aws_instance.web`	Servidor EC2 que instala Apache al arrancar (`user_data`).

Detalles que evitan errores:

- **Egress:** Terraform elimina la regla de salida que AWS crea por defecto en un security group. Si no declaras `egress`, tu servidor no puede salir a internet.

- **AMI:** los IDs de imagen cambian por región y se retiran con el tiempo. Por eso los buscamos con un `data "aws_ami"` en lugar de escribirlos fijos.
- **Capa gratuita:** revisa en la consola EC2 qué tipo dice "Free tier eligible" en tu cuenta (`t2.micro` en cuentas antiguas, `t3.micro` en cuentas nuevas).

Comando	Qué verás
`terraform init`	Descarga `hashicorp/aws` (es grande, puede tardar).
`terraform plan`	`Plan: 7 to add, 0 to change, 0 to destroy.`
`terraform apply`	Crea los recursos en orden e imprime los outputs, entre ellos `web_url`. Espera 1-2 minutos y ábrela en el navegador.
`terraform state list`	Los 7 resources + el data source.
`terraform destroy`	Borra todo en orden inverso: primero la EC2, al final la VPC.

5. Modificar recursos existentes

Cambiamos los tags del servidor en el código:

```
tags = {
  Name        = "edutek-web-prod"   # cambiado!
  Environment = "production"        # nuevo!
}
```

`terraform plan` responde `~ update in-place` y `Plan: 0 to add, 1 to change, 0 to destroy`. La instancia es la misma (mismo ID, misma IP): solo se actualizan sus etiquetas.

Símbolo	Significado	Ejemplo
`+`	Crear	Un recurso nuevo en el código.
`~`	Modificar en sitio	Cambiar tags: el recurso sigue siendo el mismo.
`-/+`	Destruir y volver a crear	Cambiar la AMI de una EC2 (`# forces replacement`): nace otra instancia, con otra IP.
`-`	Destruir	Borrar un recurso del código o usar `terraform destroy`.

> **Para recordar**
>
> **Lee siempre el plan antes de aplicar.** Un `~` es inofensivo; un `-/+` sobre una base de datos de producción puede significar perder datos.

6. Drift detection

Drift es cuando la realidad se aleja del código porque alguien hizo un cambio fuera de Terraform. Lo simulamos así:

- En la consola de AWS → EC2 → pestaña Tags, cambiamos el `Name` a `hacked-server`.
- En la terminal ejecutamos `terraform plan`. Terraform consulta AWS, detecta la diferencia y propone corregirla:

```
~ resource "aws_instance" "web" {
    ~ tags = {
        ~ "Name" = "hacked-server" -> "edutek-web-prod"
      }
  }
Plan: 0 to add, 1 to change, 0 to destroy.
```

- Con `terraform apply` el tag vuelve al valor del código.

El código es la fuente de verdad, el state es la memoria y `terraform plan` es tu red de seguridad: compara ambos con la realidad y te muestra qué va a cambiar **antes** de tocar nada.

7. Ejercicio estrella

- Replica el lab completo en tu cuenta de AWS.
- Agrega un recurso extra: `aws_s3_bucket`.
- Provoca un drift (por ejemplo, cambia un tag en la consola) y detéctalo con `terraform plan`.
- **IMPORTANTE:** ejecuta `terraform destroy` al terminar. Lo que quede encendido en AWS se cobra.

```
resource "aws_s3_bucket" "lab" {
  bucket_prefix = "edutek-clase3-"   # AWS agrega un sufijo único
  force_destroy = true               # permite borrarlo aunque tenga archivos

  tags = {
    Name = "edutek-bucket"
  }
}
```

Ojo: el nombre de un bucket es único en **todo AWS** (no solo en tu cuenta). Si usas `bucket = "mi-bucket"` es muy probable que recibas `BucketAlreadyExists`. Solo se permiten minúsculas, números, guiones y puntos.

8. Comandos nuevos de esta clase

Comando	Para qué sirve
`terraform graph`	Muestra el grafo de dependencias (formato DOT, se dibuja con Graphviz).
`terraform state list`	Lista los recursos que hay en el state.
`terraform state show <dirección>`	Muestra todos los atributos de un recurso, ej. `terraform state show aws_instance.web`.
`terraform output`	Vuelve a mostrar los outputs sin hacer apply.
`terraform apply -auto-approve`	Aplica sin pedir confirmación (úsalo solo en labs).

> **Key takeaways**
>
> **Providers** son los drivers que conectan Terraform con la API de cada nube.
>
> **Resources** se identifican como `tipo.nombre` y se referencian entre sí, creando dependencias.
>
> El **state** es sagrado: es la memoria de tu infraestructura. En equipo, siempre en un backend remoto.
>
> `terraform plan` detecta el drift y te muestra los cambios **antes** de aplicarlos.